

1. Compare Single-Core (sequential) Sorting and Multi Core Sorting.

a) Performance Improvement.

Single Core Sorting

- * Uses only one CPU core for sorting.
- * Operations are performed sequentially.
- * Suitable for small and moderate-sized datasets.

Multi Core Sorting.

- * Uses multiple CPU cores simultaneously.
- * The dataset can be divided into smaller parts and processed in parallel.
- * Can significantly reduce execution time for large datasets.

b) Synchronization overhead.

Single-Core Sorting:

- * Has very little synchronization overhead.
- * Only one thread is normally executing the sorting operation.
- * No need to coordinate multiple threads.

Multi-core Sorting:

- * Requires Synchronization between multiple threads.

- * Threads may need to communicate or combine their results.

- * Synchronization increases overhead.

(c) practical challenges.

Multi-Core sorting has several practical challenges:

1. Dividing the dataset efficiently among different CPU cores.
2. Managing Multi threads
3. Synchronizing the threads.
4. Avoiding race conditions.

Conclusion:

Multi-Core sorting is generally better for large datasets because multiple CPU cores can process different parts of the data

simultaneously.

2. Compare time complexity and space complexity

a) Trade-offs.

Time complexity describes how the execution time of an algorithm increases as the input size increases.

Example include:

- $O(1)$ - Constant time.
- $O(\log n)$ - Logarithmic time.
- $O(n)$ - Linear time.
- $O(n \log n)$ - Linearithmic time.
- $O(n^2)$ - Quadratic time.

space complexity: describes how much additional memory an algorithm requires as the input size increases.

Example include:

- $O(1)$ - Constant extra space.
- $O(\log n)$ - Logarithmic extra space.
- $O(n)$ - Linear extra space.

There is often a trade-off between time and space.

* An algorithm may use more memory to reduce execution time.

* An algorithm may use less memory but take more time.

b) situations where one is prioritized

Time complexity is prioritized when:

1. fast execution is required
2. The system processes very large datasets.
3. Quick response is important.
4. The application is time-sensitive

Example: realtime applications, large-scale data.

Space complexity is prioritized when:

1. Available memory is limited.
2. The application runs on a memory (or) trained device.
3. The dataset is large but ~~space~~ RAM is limited.

Example: embedded system, mobile devices

~~Conclusion:~~

~~Time~~ Time complexity is prioritized when speed, and fast execution are important, whereas space complexity is prioritized when memory is limited.